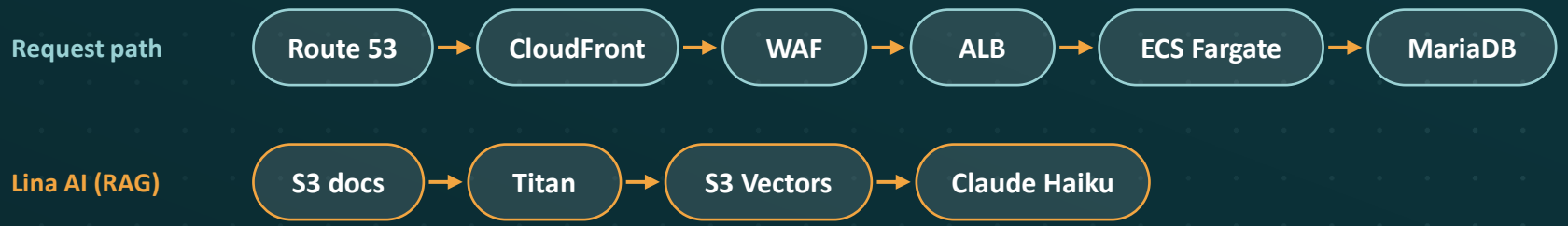
FINAL CAPSTONE PRESENTATION · OCTOBER 2026

LangNation Cloud Platform

Secure, highly available, containerized learning platform with AI/RAG on AWS



Presented by

SERAJ AHMAD SIDDIQUI

AWS Cloud Computing with AI Bootcamp

Guidance: Micheal (Theory Teacher) · Rahul Patel (Practical Teacher) · Dominic Singaraj (HoD)

12 milestones

implemented, validated and documented

2 AZs

multi-AZ app tier + MariaDB database

Grounded AI

Lina answers from LangNation knowledge

Live endpoint

www.langnation.com

The challenge and the final design goal

Overview

The capstone grew from a simple container into a recoverable, observable and AI-enabled platform.



CHALLENGE 01

Disposable sandbox

AWS resources are cleaned overnight, so reproducibility is a real requirement.



CHALLENGE 02

Security

No direct ECS or RDS exposure; CloudFront + WAF + JWT authentication.



CHALLENGE 03

Availability

Two AZs, two baseline ECS tasks, Auto Scaling and a Multi-AZ database.



CHALLENGE 04

AI grounding

Lina must answer from LangNation knowledge, not generic model memory.



CHALLENGE 05

Release quality

Bad code or vulnerable images must be blocked before ECS deployment.



CHALLENGE 06

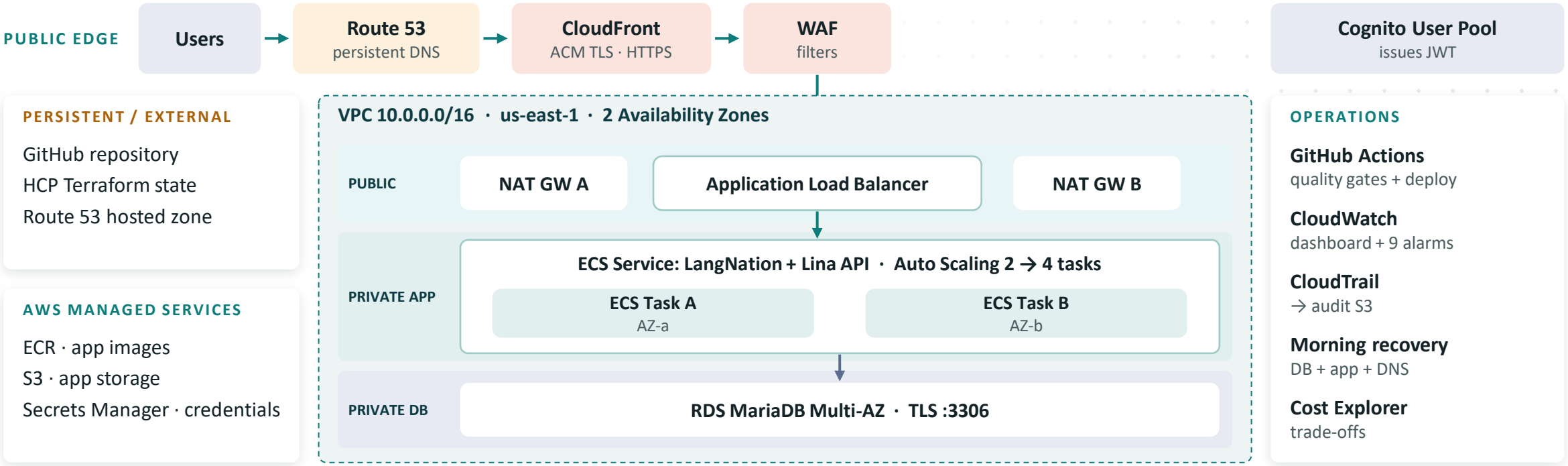
Cost awareness

Production-like resilience has a real fixed cost; trade-offs must be explicit.

Final master architecture — Milestones 1–12

M1–M12

One connected view of public edge, private runtime, data, event processing, RAG, automation and governance.



DOCUMENT PROCESSING + RAG

S3 docs → EventBridge → SQS + DLQ → Lambda → Titan Embeddings → S3 Vectors → Claude Haiku

Lina runtime: question → Titan query vector → S3 Vectors → Claude Haiku → grounded answer + source

Why I chose these AWS resources

Design

Every service answers a design goal: secure, highly available, reproducible, grounded or observable.

NETWORK & EDGE
VPC · 2 AZs
Isolates tiers; two AZs remove a single point of failure

NETWORK & EDGE
NAT Gateways ×2
Outbound-only internet for private tasks, one per AZ

NETWORK & EDGE
Route 53
Persistent DNS that survives sandbox cleanup

NETWORK & EDGE
CloudFront + ACM
One HTTPS entry point with managed TLS

NETWORK & EDGE
AWS WAF
Blocks common web attacks at the edge

COMPUTE
Amazon ECR
Private images tagged by Git SHA — traceable releases

COMPUTE
ECS Fargate
Private containers with no servers to manage

COMPUTE
ALB + Auto Scaling
Health checks across both AZs; scales 2 → 4 tasks

DATA
RDS MariaDB Multi-AZ
Managed DB with standby, TLS, encryption, backups

DATA
Amazon S3
Private, versioned storage for files and documents

SECURITY
Secrets Manager
Database credentials never stored in code

SECURITY
Amazon Cognito
Managed sign-in; the API only verifies JWTs

EVENTS & AI
EventBridge · SQS
Durable async processing into Lambda, with retries + DLQ

EVENTS & AI
Bedrock: Titan + Haiku
Managed embeddings and answers — no model hosting

EVENTS & AI
S3 Vectors
Serverless vector index — no vector database to run

OPERATIONS
CloudWatch + CloudTrail
Alarms, dashboard and a validated audit trail

Architecture evolution — 12 milestones

Journey

Each milestone added one capability while preserving the previous architecture.

M1 Network foundation ✓
VPC · 6 subnets · 2 NAT · SG

M2 Container runtime ✓
Docker · ECR · ECS Fargate

M3 Availability ✓
ALB · 2 AZs · Auto Scaling

M4 Deployment ✓
GitHub Actions · OIDC · SHA

M5 Data layer ✓
MariaDB · S3 · Secrets

M6 Public edge ✓
Route 53 · CF · WAF · Cognito

M7 Events ✓
EventBridge · SQS · Lambda · DLQ

M8 RAG ingestion ✓
Titan embeddings · S3 Vectors

M9 Real application ✓
58-table restore · Lina runtime

M10 Operations ✓
9 alarms · CloudTrail audit

M11 Quality gates ✓
Tests · tf validate · Trivy

M12 Resilience & cost ✓
Self-healing · Cost Explorer

Foundation | Data & edge | Events & AI | Operations & quality ✓ implemented and validated

Milestone 1 — Foundation & networking

M1

The network is split by trust level, Availability Zone and routing behavior.

VPC 10.0.0.0/16

	Availability Zone us-east-1a	Availability Zone us-east-1b
Public internet-facing	**10.0.1.0/24** ALB · NAT Gateway	**10.0.2.0/24** ALB · NAT Gateway
Private app compute	**10.0.11.0/24** ECS Fargate · no public IP	**10.0.12.0/24** ECS Fargate · no public IP
Private DB data	**10.0.21.0/24** RDS only · local route table	**10.0.22.0/24** RDS only · local route table

Routing & security

Public	→ Internet Gateway
Private app	→ same-AZ NAT
Private DB	→ local only
SG path	→ ALB → ECS → DB

No public IP on ECS
tasks are reachable only through the ALB

No public DB route
Terraform / HCP defines every route

Milestones 2–3 — Containers & scaling

M2–M3

Private Fargate tasks behind a load balancer — health-checked, spread across AZs and scaled by CPU.



Request path: CloudFront → ALB → target group → healthy task

Image path: build → ECR → rolling deploy

2	2 AZs	2 → 4	50%	200
baseline tasks	task distribution	Auto Scaling range	CPU target	/health expected

Controlled validation

Temporarily lowered the CPU target to 1%; generated load scaled the service from 2 to 4 tasks — then restored the normal 50% target.

Ownership lesson

Terraform defines the service and scaling configuration. Application Auto Scaling owns the live desired task count.

Milestone 4 — Automated deployment

M4

Application delivery is traceable, short-lived and independently validated.

Release pipeline — GitHub Actions

1. Push to main
2. GitHub Actions
3. OIDC → STS
4. Docker build
5. ECR · Git SHA
6. New task revision
7. ECS rolling deploy
8. Verify final state

No long-lived AWS keys

GitHub proves its identity through OIDC; AWS STS issues temporary credentials for the restricted deployment role.

Immutable deployment

Docker image tag = Git commit SHA — a direct link from source code to ECR image to ECS task revision.

Final-state validation

Wait for services-stable, verify the exact image SHA, desired = running, pending = 0 and public /health.

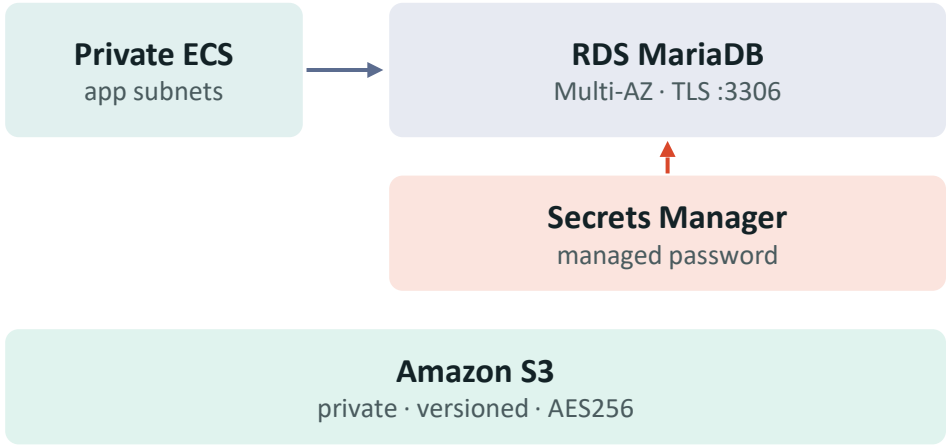
Ownership: Terraform → infrastructure · GitHub Actions → live task-definition revision · Auto Scaling → live desired count

Milestones 5–6 — Data, edge & authentication

M5–M6

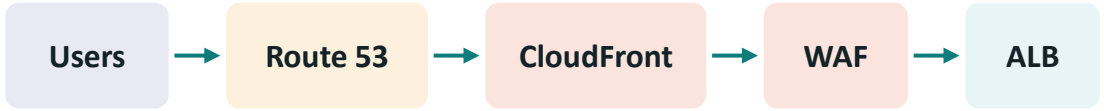
The database stays private; the public path is HTTPS → CloudFront/WAF → ALB → ECS.

DATA LAYER · M5

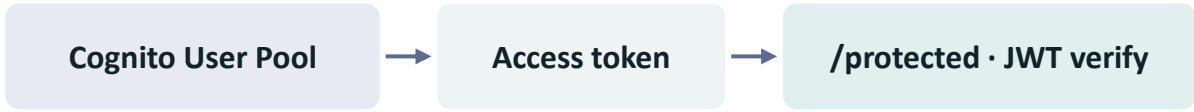


- MariaDB 11.8.9 · db.t3.micro
- 20 GiB gp3 · max 100 GiB
- 7-day backups · public access disabled
- S3 HTTPS-only policy + lifecycle controls

PUBLIC EDGE + AUTH · M6



- ACM terminates viewer TLS at CloudFront
- ALB :80 accepts only CloudFront origin-facing addresses



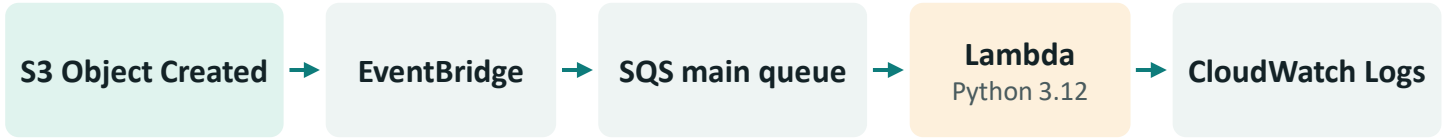
200	401	200	403
/health public	**no token** rejected	**valid token** allowed	**WAF test** blocked

Milestone 7 — Serverless & event-driven processing

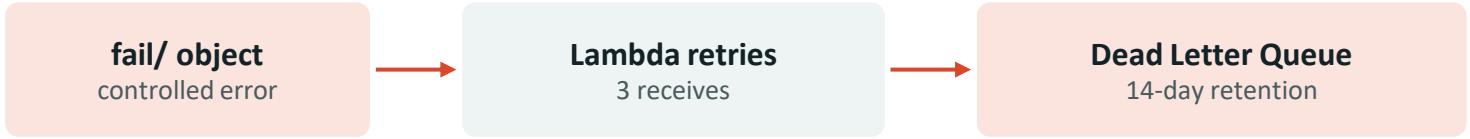
M7

S3 uploads become durable asynchronous work with retries and a Dead Letter Queue.

NORMAL PATH



FAILURE PATH



Queue settings: visibility timeout 180 s · maxReceiveCount 3 · DLQ retention 14 days

Why SQS before Lambda? Durable buffering, decoupling and explicit retry / DLQ behavior — nothing is lost if Lambda fails.

Validated normal path

An Object Created event reached Lambda and produced CloudWatch logs.

Validated failure path

An intentional fail/ object was retried 3 times, then moved to the DLQ.

Key lesson

Serverless still needs explicit retry, queue and failure semantics.

Milestone 8 — RAG knowledge pipeline

M8

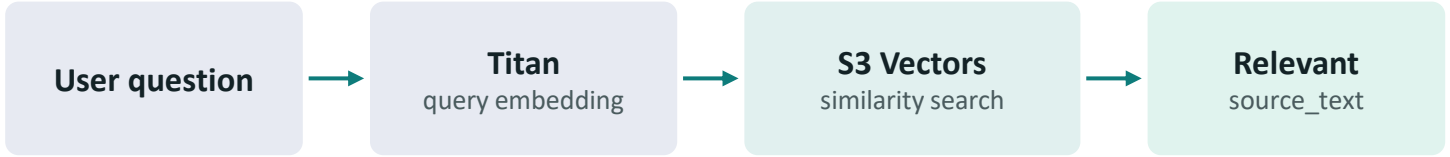
Documents are converted into semantic vectors that can be searched by meaning.

INGESTION



1024 dimensions · float32 · cosine distance · source metadata on every vector

RETRIEVAL



Retrieval proof

Relevant LangNation knowledge was ranked ahead of an unrelated document.

Why Titan for both?

Documents and questions must live in the same semantic embedding space, or the search can’t compare them.

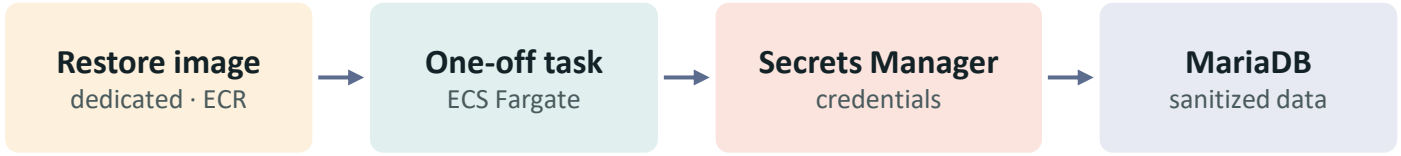
Lesson: RAG starts with retrieval quality — ingest correctly → embed consistently → prove semantic ranking.

Milestone 9 — Real application & Lina RAG runtime

M9

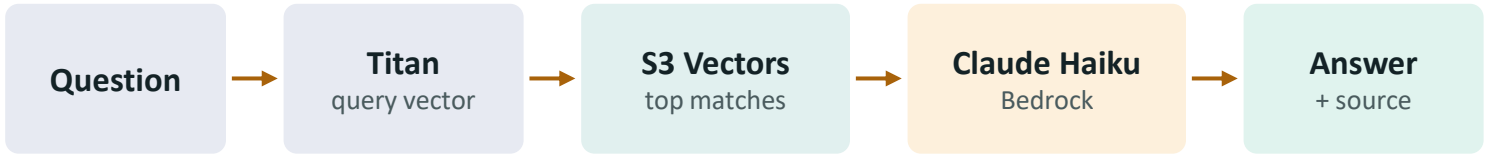
The capstone moved from a demo API to sanitized real application data and grounded AI answers.

DATABASE RECOVERY



58 tables restored · /api/health → database = true over verified RDS TLS

LINA RUNTIME



Response includes grounded = true and the retrieved LangNation source metadata.

58
tables restored

true
database health over TLS

grounded
answers with sources

Key fix

RDS required secure transport. I updated the application for verified TLS instead of weakening the database.

Milestone 10 — Monitoring, security & governance

M10

Operational visibility and audit evidence were added around the full platform.

Amazon CloudWatch

9
custom alarms

1
operations dashboard

Health, performance and failure signals — including the M7 Dead Letter Queue.

AWS CloudTrail & audit bucket

- Multi-Region trail
- Global service events
- Management events
- Log-file validation
- Private AES256 + versioned bucket
- 30-day lifecycle

Alarm proof

Existing DLQ test messages triggered a real ALARM state; after purging the queue the alarm returned to OK.

CloudTrail governance

Management activity across all Regions is recorded, including global services.

Audit storage

A real .json.gz audit log was delivered to the private, encrypted, versioned bucket.

Milestone 11 — CI/CD quality gates

M11

Release hardening: nothing ships until application, Terraform and container checks all pass.

Push to main
code · Terraform · Dockerfile

`quality-gates.yml` · all three must pass

Application gate

- Node.js 24
- npm ci (locked deps)
- npm run check
- npm audit --omit=dev

Infrastructure gate

- Terraform 1.16.2
- fmt -check -recursive
- init -backend=false
- validate (environments/dev)

Container gate

- docker build (node:24-alpine)
- run + /health smoke test
- Trivy image scan
- no CRITICAL findings

`deploy.yml`

1. Calls quality gates first
2. Build & push to ECR
3. Deploy ECS (Fargate)
4. Serve via CloudFront

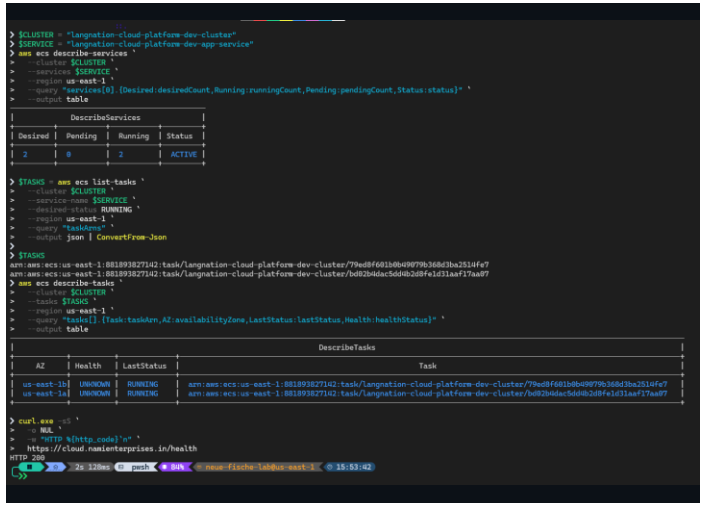
Real security outcome

Trivy blocked a CRITICAL vulnerability in the tar package of the old base image. I fixed it by upgrading the production base image to Node 24 — the gate was not weakened.

Milestone 12 — Resilience tested through failure

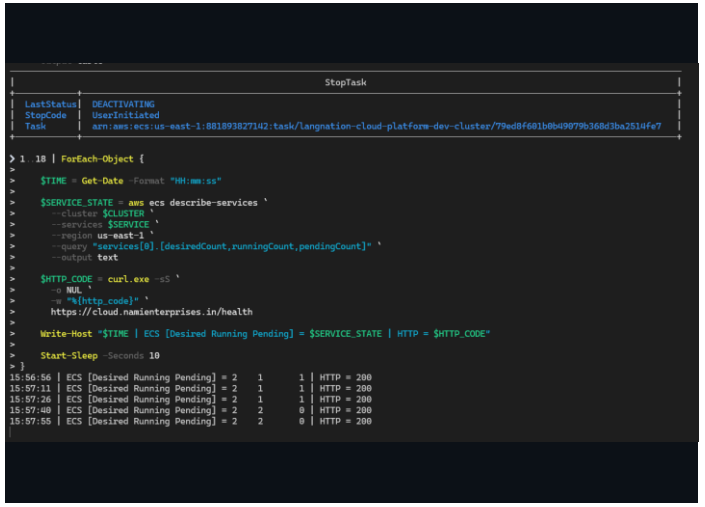
M12

I intentionally stopped a running ECS service task and observed the actual recovery.



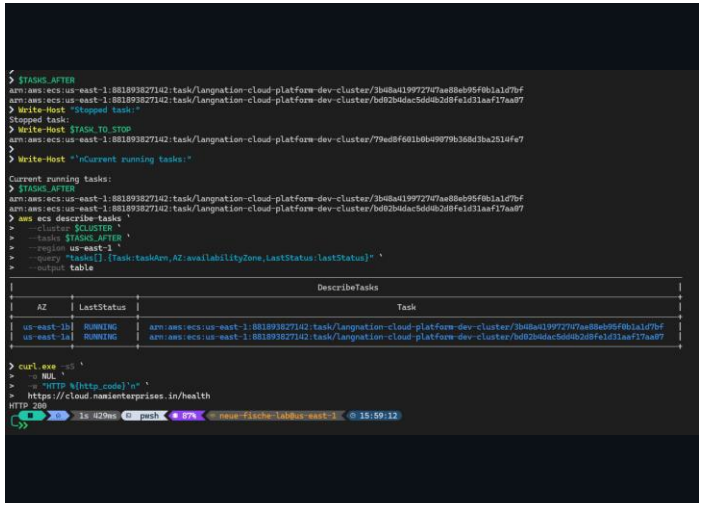
1 · Baseline

2 desired / 2 running / 0 pending
Tasks across both AZs · HTTP 200



2 · Controlled stop

2 desired / 1 running / 1 pending
Public endpoint still HTTP 200



3 · Recovery

Replacement task healthy
Final state 2 / 2 / 0

≈ 44 seconds observed recovery back to the two-task baseline — while the public endpoint kept answering HTTP 200.

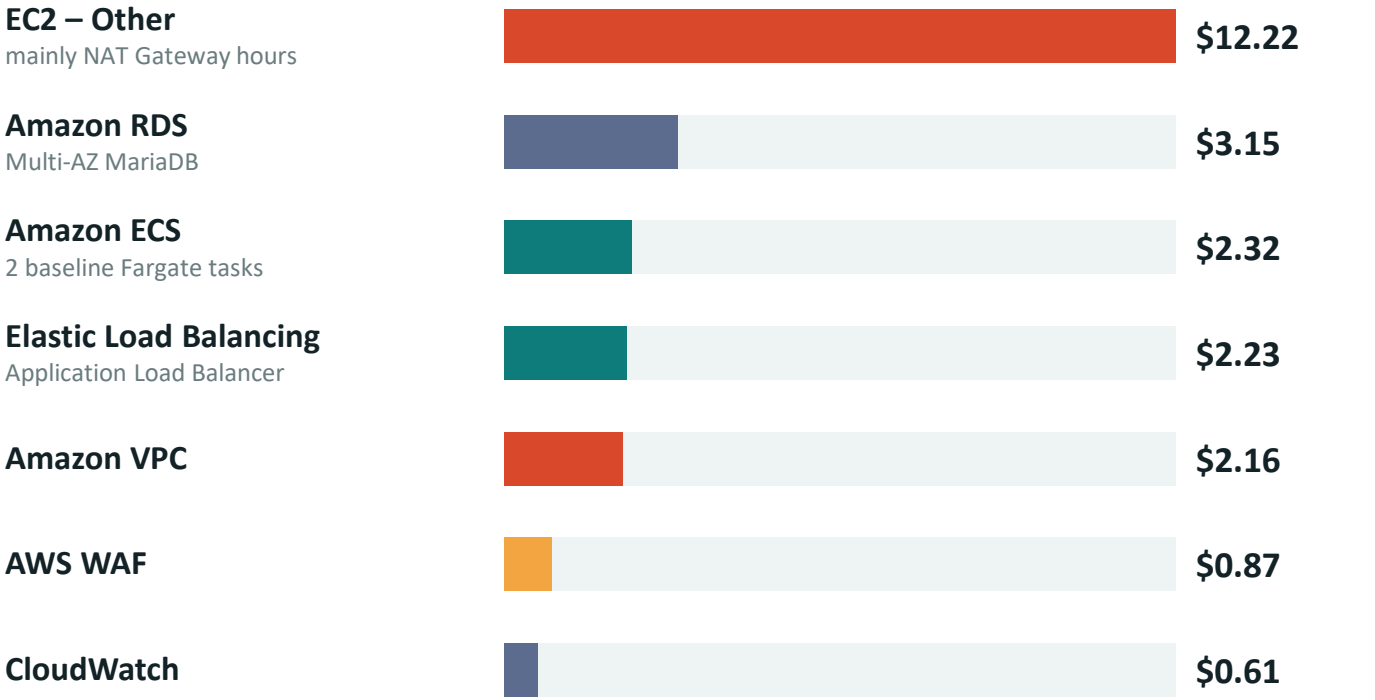
Real AWS cost analysis — October 2026

M12 Cost

Cost Explorer was used to find actual service and usage-type drivers, not theoretical list prices.

Cost by service

USD



Decision: document cheaper dev options — don't weaken the final architecture.

$28.07

total incl. shown tax

$23.78 usage subtotal + $4.29 tax

Key finding

The largest driver was NAT Gateway hourly runtime — not high traffic volume.

Why I kept the architecture

Two NAT Gateways, Multi-AZ RDS and two baseline ECS tasks preserve the resilience story.

Dev-only options

Single NAT, Single-AZ RDS, scheduled non-production runtime, careful IPv4 reduction.

OP01 — Automated morning sandbox recovery

Operations

After Auto sandbox activation, recovery coordinates infrastructure, data, application and DNS.

07:37
primary · Europe/Berlin

08:17
fallback · Europe/Berlin

Auto sandbox activation first

PHASE 1 · INFRASTRUCTURE

GitHub Actions
scheduled run → **HCP Terraform API**
recovery run → **Plan + auto-apply**
langnation-capstone-dev → **Infrastructure**
recreated

PHASE 2 · DATA, APPLICATION & DNS

Build image
OIDC · Git SHA → ECR → **Restore DB**
one-off task · 58 tables → **Deploy ECS**
new revision · stable → **Sync DNS**
CloudFront → Route 53 → **GET /health**
→ READY

Normal runs stay manual

Regular Terraform changes still go through the standard approval.

Recovery auto-applies

Only the morning recovery uses a dedicated auto-apply run.

No long-lived keys

GitHub OIDC + AWS STS, short-lived credentials.

Dynamic DNS

CloudFront domain read from Terraform outputs, synced to Route 53.

Security and ownership model

Design

The platform stays maintainable because each system has a clear trust boundary and owner.

Public edge

- Route 53 persists outside the sandbox
- CloudFront terminates HTTPS
- WAF filters requests
- ALB accepts CloudFront origin only

Private runtime

- ECS tasks have no public IP
- DB sits in private DB subnets
- ECS → DB only on 3306
- Secrets Manager supplies credentials

Identity

- GitHub OIDC → temporary STS credentials
- No static AWS deployment keys
- Cognito issues access tokens
- JWT protects application routes

Ownership

- HCP Terraform → infrastructure & state
- GitHub Actions → releases
- Auto Scaling → live desired count
- Recovery workflow → rebuild, data, DNS

Core security path: Internet → CloudFront / WAF → ALB → ECS → MariaDB

Evidence — tested, not just configured

Validation

The project includes controlled success, failure and recovery validation across the stack.

HTTP 200

Public /health through CloudFront

WAF 403

Controlled malicious-style request blocked

ALB blocked

Direct origin bypass prevented

JWT 401 → 200

Protected API without / with token

DLQ retry

Controlled Lambda failure reached the DLQ

RAG retrieval

Relevant knowledge ranked first

ALARM → OK

DLQ backlog detected, then cleared

Trivy block

CRITICAL vulnerability prevented release

2/1/1 → 2/2/0

ECS self-healed while the endpoint stayed up

Engineering challenges I solved

Lessons

Real troubleshooting — each symptom traced to a root cause and fixed permanently.

01 Nightly sandbox cleanup

CAUSE Disposable training environment

FIX Terraform / HCP rebuild + OP01 recovery

02 OIDC role assumption denied

CAUSE Trust policy did not match the immutable GitHub identity

FIX Tightened owner / repo / branch / audience conditions

03 Database health false

CAUSE RDS required secure transport

FIX Verified RDS TLS in the real application

04 DLQ alarm fired immediately

CAUSE Controlled M7 test messages were still queued

FIX Explained the backlog, purged, observed ALARM → OK

05 Trivy blocked deployment

CAUSE CRITICAL vulnerable tar package in the old base image

FIX Upgraded the image to Node 24; kept the gate strict

06 Cost seemed to be “EC2 – Other”

CAUSE The billing category hid networking usage

FIX Drilled into usage type — found NAT Gateway hours

Final architecture decisions & trade-offs

Decisions

I optimized for a defensible production-style design, not the lowest possible sandbox bill.

Decision	Benefit	Trade-off	Final choice
2 NAT Gateways	Same-AZ private egress, less AZ dependency	Higher fixed hourly cost	**Keep; single NAT is a dev-only option**
Multi-AZ MariaDB	Database failover and availability	Higher DB runtime cost	**Keep; Single-AZ only for cost-controlled dev**
2 baseline ECS tasks	Availability + rolling-deployment headroom	More Fargate cost	**Keep; needed for a meaningful resilience proof**
Persistent Route 53	Stable public DNS across sandbox rebuilds	Cross-account recovery step	**Keep; enables a disposable workload account**
Strict Trivy gate	Blocks vulnerable releases	Can stop a deployment unexpectedly	**Keep; fix the root cause, never bypass the scan**

Key takeaway

The final result is not a collection of AWS services. It is one platform that can be rebuilt, deployed, secured, observed, grounded with RAG, tested through failure — and explained with evidence.

Secure by design

Private compute and data · WAF · OIDC · Cognito

Highly available

Multi-AZ DB · 2 AZs · 2 tasks · Auto Scaling

Grounded AI

Titan embeddings · S3 Vectors · Claude Haiku

Operational

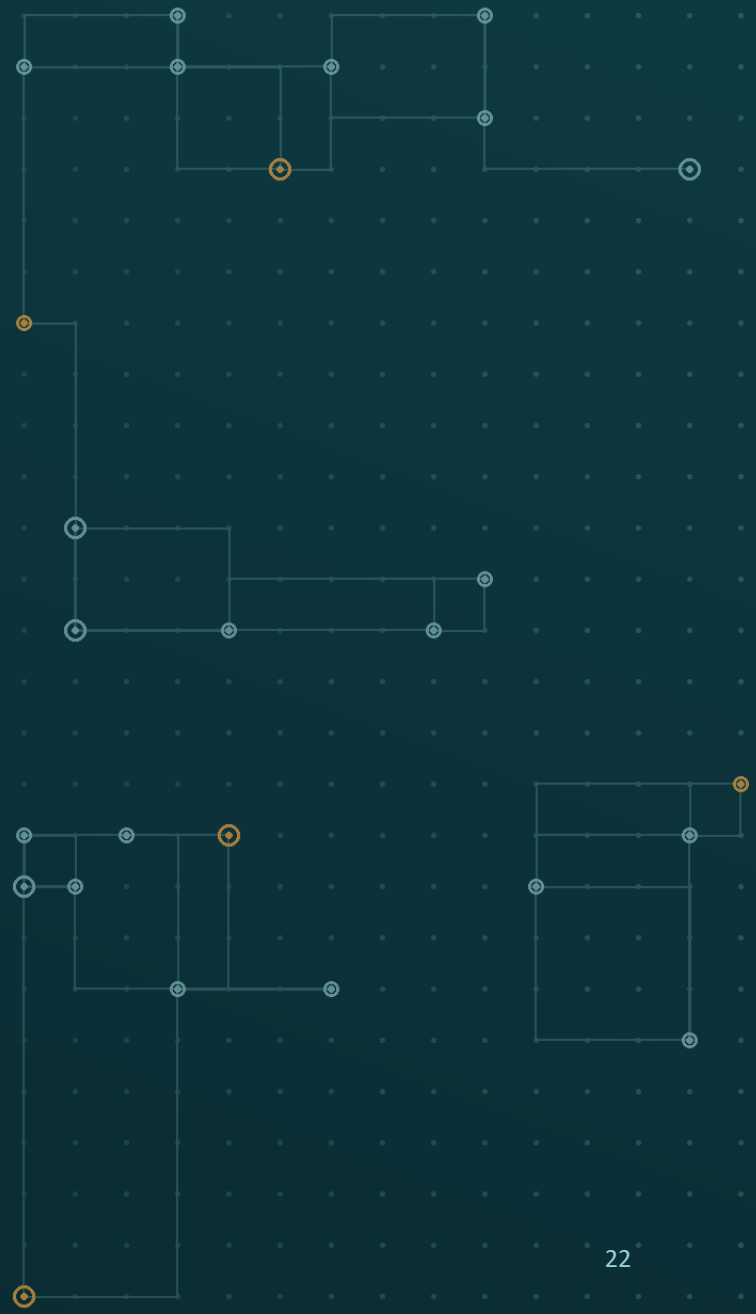
CloudWatch · CloudTrail · OP01 recovery

Quality controlled

Tests · Terraform validation · smoke test · Trivy

“Built from code. Protected at the edge.
Grounded by knowledge. Proven through failure.”

cloud.namienterprises.in



Source code, infrastructure & documentation

Repository

The repository is organized so architecture, implementation, evidence and operations are easy to review.

```
langnation-cloud-platform/
├── application/                real Node.js app + Lina runtime
├── database/                   sanitized seed / restore assets
├── knowledge/                  RAG source documents
├── lambda/                     document processor
├── terraform/
│   ├── environments/dev/       composition + seed objects
│   └── modules/                network, security, container, data, edge, RAG…
├── .github/workflows/          quality gates, deploy, recovery
└── docs/
    ├── milestones/             M01 → M12
    ├── evidence/               proof + screenshots
    ├── architecture/           milestones, flows, master
    └── operations/             OP01 recovery runbook
```



Scan to open on GitHub

github.com/sheikhseraj/langnation-cloud-platform

Terraform + HCP · GitHub Actions
M1–M12 docs · Evidence + OP01

Acknowledgements & support

Thanks

AWS Cloud Computing with AI Bootcamp · guidance, feedback and peer support behind this capstone.

Instruction & leadership

Micheal — Theory Teacher
Rahul Patel — Practical Teacher
Dominic Singaraj — Head of Department

Supporting teachers

Shahabuddin · Charisima · Pushpajit · Siranach
and the wider teaching team

Classmates & peers

Discussions, feedback, testing, troubleshooting and motivation throughout the capstone



Questions?

Thank you for the questions and feedback that helped improve the architecture.

cloud.namienterprises.in

github.com/sheikhseraj/langnation-cloud-platform

Key technical configuration

BACKUP

Exact values for Q&A — not a main presentation slide.

VPC	10.0.0.0/16
Public subnets	10.0.1.0/24 · 10.0.2.0/24
Private app	10.0.11.0/24 · 10.0.12.0/24
Private DB	10.0.21.0/24 · 10.0.22.0/24
ECS	Fargate · 256 CPU · 512 MiB · port 3000
Scaling	min 2 · max 4 · CPU target 50%
RDS	MariaDB 11.8.9 · db.t3.micro · Multi-AZ
RDS storage	20 GiB gp3 · max 100 GiB · 7-day backups
S3	private · versioned · AES256 · HTTPS-only
Event pipeline	SQS visibility 180 s · maxReceiveCount 3 · DLQ 14 days
Embeddings	Titan Text Embeddings V2 · 1024 dims
Chunking	200 words · 40-word overlap
CloudWatch	dashboard + 9 custom alarms
CloudTrail	multi-Region · management events · log validation
Domain	cloud.namienterprises.in
M12 recovery	~44 s observed back to 2/2/0

FAQs — likely questions & short answers

BACKUP

Use as a mental checklist during Q&A.

Why two NAT Gateways?
Same-AZ private egress and less AZ dependency; single NAT is a dev cost option.

Why persistent Route 53?
The sandbox is disposable — DNS must survive CloudFront and ACM recreation.

Why OIDC instead of AWS keys?
Short-lived federated credentials; no permanent AWS secrets in GitHub.

Why block direct ALB access?
Otherwise users could bypass CloudFront and WAF using the ALB DNS name.

Why SQS before Lambda?
Durable buffering, decoupling and explicit retry / DLQ behavior.

Why Titan for documents and queries?
Both vectors must live in the same semantic embedding space.

Why MariaDB TLS?
RDS required secure transport; the app complies instead of weakening the DB.

How do you prove Lina is grounded?
The response includes grounded = true and the retrieved source metadata.

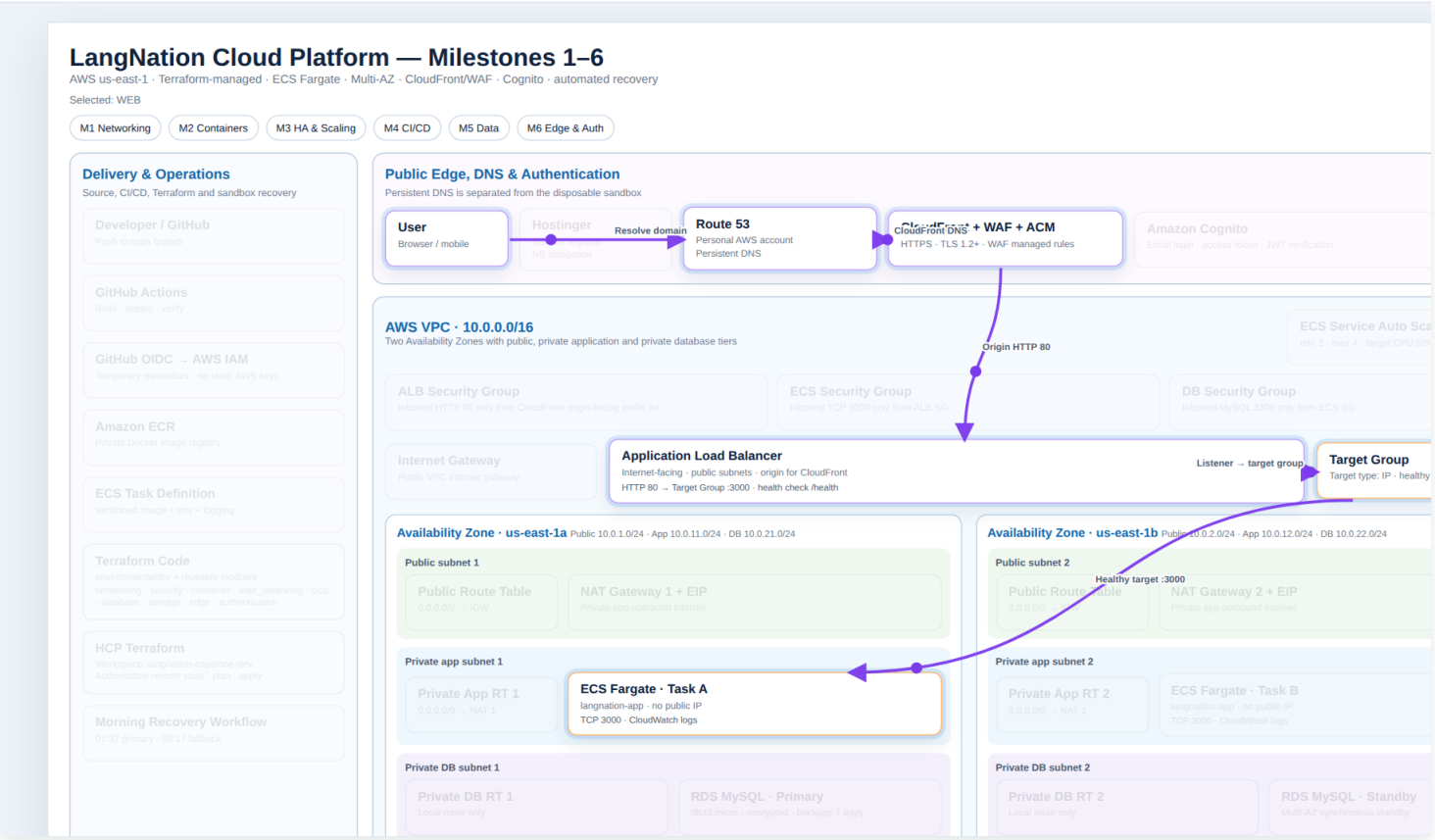
Why did Trivy stop deployment?
A real CRITICAL vulnerability existed; the image was fixed, not the gate.

What happens after nightly cleanup?
Activation → HCP rebuild → DB restore → app deploy → DNS sync → health check.

Interactive walkthrough — core platform (M1–M6)

BACKUP

An animated view of the foundation milestones — choose a milestone or flow and watch the request path move.



Covers Milestones 1–6 (network, containers, deployment, data, edge and authentication).

Open the live walkthrough

Opens the HTML file in your browser. Keep it in the same folder as this deck.

Keyboard

`1–6`	select a milestone
`A`	all flows together
`S`	auto sequence
`Space`	play / pause
`F`	fullscreen